Implementación — Módulo de Subastas y Puja en Vivo (3ª Entrega)

> **Alcance:** Frontend Android (Jetpack Compose), datos **mock** en memoria. El backend será reemplazado por Supabase; toda la capa de datos está aislada detrás de interfaces para poder cambiarla sin tocar UI ni ViewModels.

---

1. Resumen

Se incorporó el flujo completo de **Subastas** al tab **Inicio**:

```
Subastas activas  ⟶  Detalle de la pieza  ⟶  Subasta en vivo (puja)
```

Cubre las reglas del TPO: visibilidad según invitado/registrado, gating por categoría y medio de pago, rango de puja (+1% / +20% del valor base), exención de límites para ORO/PLATINO, moneda por subasta (ARS/USD), pujas en tiempo real y estados ganando / superado / oferta ganadora.

Validado end-to-end en emulador (Pixel_10). Tests unitarios de reglas: **10/10 OK**.

---

2. Pantallas y funcionalidades

2.1 Subastas activas (`AuctionListScreen`)

- Lista de subastas con imagen, badges de **estado** (EN VIVO / PROGRAMADA / FINALIZADA) y **categoría**, rematador y horario.
- Filtros por categoría (Todas / Común / Especial / Plata / Oro / Platino).
- **Invitado:** el precio se oculta → "Iniciá sesión para ver".
- **Registrado:** muestra la oferta actual con la moneda de la subasta (`$15.500` ARS / `US$85.000` USD).

2.2 Detalle de la pieza (`CatalogItemDetailScreen`)

- Carrusel de imágenes, número de pieza, título, descripción, dueño actual.
- **Precio base** visible solo para usuarios registrados.
- Soporte de **pieza multi-elemento** ("compuesta por N elementos").
- **Historial de la pieza** (artista/diseñador + fecha + historia) expandible para obras de arte.
- Botón "Ver subasta en vivo".

2.3 Subasta en vivo (`LiveAuctionScreen` + `LiveAuctionViewModel`)

- **Artículo actual**, **oferta actual** y **rango de puja válido**.

- **Historial de pujas** en tiempo real, marcando la oferta más alta y la propia ("(vos)").
- Campo de monto + botón **PUJAR**.
- **Gating de puja** (regla TPO): se puede pujar solo si el usuario está registrado **y** su categoría habilita la subasta **y** tiene medio de pago verificado. Si no, la pantalla queda en **solo lectura** con el motivo.
- **Una puja a la vez:** al pujar, el campo se bloquea y se muestra "Esperando confirmación del sistema..." hasta confirmar.
- **Estados de puja:**
- 🟢 GANANDO — "Vas ganando la subasta"
- 🔴 SUPERADO — "Te superaron la oferta"
- 🟡 GANADA — "¡OFERTA GANADORA! Ganaste la pieza"
- ⚪ VIEW_ONLY — motivo del bloqueo
- **Pujas de la competencia** simuladas en tiempo real (mock del futuro WebSocket).

---

3. Reglas de negocio (`BidRules`)

Regla	Fórmula / comportamiento
Puja mínima	`ofertaActual + 1% · valorBase`
Puja máxima	`ofertaActual + 20% · valorBase`
Categorías ORO / PLATINO	**Exentas** de límites: solo deben superar la oferta actual (sin máximo)
Monto inválido	nulo / ≤ 0 → rechazado
Acceso por categoría	jerarquía COMÚN < ESPECIAL < PLATA < ORO < PLATINO (`puedeAcceder`)

Ejemplo del enunciado: base 10.000, última 15.000 → mínimo **15.100**, máximo **17.000**.

---

4. Arquitectura (capas)

```
ui/auction        → Pantallas Compose + ViewModels (estado de UI)
        |  usa
domain/BidRules    → Lógica pura de validación de pujas (testeable)
        |  usa
data/repository    → AuctionRepository (interfaz) + MockAuctionRepository
data/realtime      → BidStream (interfaz) + MockBidStream
data/model/auction → Modelos de dominio (Auction, CatalogItem, Bid, ...)
```

> Para integrar Supabase: implementar `AuctionRepository` y `BidStream` con REST/ WebSocket. La UI y los ViewModels **no cambian**.

---

5. Archivos creados

Archivo	Rol
`data/model/auction/AuctionModels.kt`	Modelos: `Categoria`, `Currency` + `formatMoney`, `Auction`, `CatalogItem`, `ArtMetadata`, `Bid`, `BidConfirmation`, `BiddingStatus`, `CurrentUser`
`domain/BidRules.kt`	Reglas de puja (min/max, exenciones, validación)
`data/repository/AuctionRepository.kt`	Interfaz del repositorio
`data/repository/MockAuctionRepository.kt`	Datos mock + estado en memoria
`data/repository/CurrentUserProvider.kt`	Usuario actual (gating)
`data/realtime/BidStream.kt`	Interfaz de stream de pujas
`data/realtime/MockBidStream.kt`	Pujas de competencia simuladas
`ui/components/CommonUi.kt`	`MoneyText`, `StatusBadge`, `CategoriaBadge`
`ui/auction/AuctionListViewModel.kt`	Estado de la lista + filtros
`ui/auction/AuctionListScreen.kt`	Pantalla "Subastas activas" + `AuctionCard`
`ui/auction/CatalogItemDetailScreen.kt`	Pantalla "Detalle de la pieza"
`ui/auction/LiveAuctionViewModel.kt`	Lógica de subasta en vivo (gating, pujas, estados)
`ui/auction/LiveAuctionScreen.kt`	Pantalla "Subasta en vivo"
`ui/auction/AuctionPreviews.kt`	`@Preview` de `AuctionCard`
`test/ … /domain/BidRulesTest.kt`	10 tests unitarios de reglas

6. Archivos modificados

Archivo	Cambio
`ui/main/MainScreen.kt`	Tab Inicio con navegación anidada lista → detalle → en vivo
`navigation/NavGraph.kt`	Entrada demo (registrado) — **temporal**
`ui/auth/LoginScreen.kt`	Botón debug "Entrar como usuario demo" — **temporal**
`data/api/RetrofitClient.kt`	`BASE_URL` → `10.0.2.2` (emulador → host)
`ui/auction/AuctionDetailScreen.kt`	**Eliminado** (mock viejo, reemplazado por `LiveAuctionScreen`)

7. Pendientes / a quitar antes de producción

- **Botón "🔧 Entrar como usuario demo"** en Login y su ruta en `NavGraph`: fueron agregados solo para validar el flujo de usuario registrado sin backend. Quitar cuando exista el login real (Supabase).

- `BASE_URL = 10.0.2.2`: válido para emulador. Para dispositivo físico, usar la IP LAN de la PC.
- Integración real de `AuctionRepository` / `BidStream` contra Supabase.

---

8. Cómo correr y validar

```
cd "TPO_DesarrolloDeAplicacionesI-master\frontend"
.\gradlew.bat :app:installDebug      # compila e instala en el emulador
.\gradlew.bat :app:testDebugUnitTest # corre los tests de BidRules
```

En la app: **Entrar como usuario demo** (o **Entrar como invitado**) → tab **Inicio** → elegir una subasta → **Ver subasta en vivo** → pujar.

Capturas de la validación: `flow_01_list_guest.png` … `flow_14_oro_usd.png`.

---

9. Código fuente completo

> Rutas relativas a `TPO_DesarrolloDeAplicacionesI-master/frontend/app/src/`.

9.1 main/java/com/subastasapp/data/model/auction/AuctionModels.kt

```kotlin
package com.subastasapp.data.model.auction

import java.util.Locale

/**
 * Categoría de un usuario o de una subasta.
 *
 * El orden de declaración define la jerarquía (COMUN es la más baja, PLATINO la más alta).
 * Un usuario puede acceder a una subasta cuando la categoría de la subasta es menor o igual
 * a la propia (regla del TPO).
 */
enum class Categoria {
    COMUN, ESPECIAL, PLATA, ORO, PLATINO;

    /** El usuario (this) puede acceder a una subasta de categoría [subasta]. */
    fun puedeAcceder(subasta: Categoria): Boolean = this.ordinal ≥ subasta.ordinal

    /** Las subastas ORO y PLATINO están exentas de los límites de % en la puja. */
    val exentaDeLimites: Boolean get() = this == ORO || this == PLATINO

    val displayName: String
        get() = name.lowercase().replaceFirstChar { it.uppercase() }
}

/** Moneda de la subasta. Una subasta es en ARS o en USD, nunca bimonetaria. */
enum class Currency(val symbol: String) {
    ARS("$"),
    USD("US$")
}

/** Formatea un importe con separador de miles estilo es-AR: 15500.0 → "$15.500". */
fun formatMoney(amount: Double, currency: Currency): String {
    val rounded = Math.round(amount)
    val withSeparators = String.format(Locale.GERMAN, "%,d", rounded) // GERMAN usa '.' como miles
    return "${currency.symbol}$withSeparators"
}

enum class AuctionStatus { PROGRAMADA, EN_VIVO, FINALIZADA }

/** Estado de la participación del usuario en la subasta en vivo. */
enum class BiddingStatus {
    /** El usuario tiene la oferta más alta. */
    GANANDO,
    /** Otro postor superó la oferta del usuario. */
    SUPERADO,
    /** La subasta cerró y el usuario ganó la pieza. */
    GANADA,
    /** El usuario puede ver pero no pujar (categoría, medio de pago o invitado). */
    VIEW_ONLY
}

/** Metadatos extra para obras de arte u objetos de diseñador. */
data class ArtMetadata(
```

```
    val artista: String,
    val fecha: String,
    val historia: String
)

/**
 * Un ítem (pieza) del catálogo de una subasta.
 * Una pieza puede estar compuesta por varios elementos (ej. "Juego de Té - 18 piezas").
 */
data class CatalogItem(
    val id: Long,
    val numeroPieza: String,
    val titulo: String,
    val descripcion: String,
    val valorBase: Double,
    val duenioActual: String,
    val imagenes: List<String> = emptyList(),
    val esMultiElemento: Boolean = false,
    val cantidadElementos: Int = 1,
    val arte: ArtMetadata? = null
)

/** Una subasta abierta o programada. */
data class Auction(
    val id: Long,
    val titulo: String,
    val categoria: Categoria,
    val estado: AuctionStatus,
    val currency: Currency,
    val rematador: String,
    val fechaHoraInicio: String,
    val itemActual: CatalogItem,
    val ofertaActual: Double = itemActual.valorBase
)

/** Una puja registrada en la subasta. */
data class Bid(
    val id: Long,
    val autor: String,
    val importe: Double,
    val timestamp: Long = System.currentTimeMillis(),
    val esGanadora: Boolean = false,
    val esMia: Boolean = false
)

/** Usuario actual (mock). Modela lo necesario para el gating de la puja. */
data class CurrentUser(
    val id: Long,
    val nombre: String,
    val categoria: Categoria,
    val tieneMedioPagoVerificado: Boolean,
    val registrado: Boolean
) {
    companion object {
        /** Invitado: no registrado, sin categoría efectiva ni medio de pago. */
        val GUEST = CurrentUser(
            id = -1,
```

```
            nombre = "Invitado",
            categoria = Categoria.COMUN,
            tieneMedioPagoVerificado = false,
            registrado = false
        )
    }
}

/** Confirmación devuelta por el servidor (mock) tras registrar una puja. */
data class BidConfirmation(
    val bid: Bid,
    val nuevaOfertaActual: Double
)
```

9.2 main/java/com/subastasapp/domain/BidRules.kt

```
package com.subastasapp.domain

import com.subastasapp.data.model.auction.Categoria
import com.subastasapp.data.model.auction.Currency
import com.subastasapp.data.model.auction.formatMoney

/**
 * Reglas de validación de pujas según el TPO. Funciones puras (testeables sin Android).
 *
 * - Puja mínima = mejor oferta actual + 1% del valor base.
 * - Puja máxima = última oferta + 20% del valor base.
 * - Las subastas ORO y PLATINO están exentas de estos límites: solo debe ser
 *   estrictamente mayor a la oferta actual.
 *
 * Ejemplo del enunciado: base 10.000, última 15.000 → rango válido 15.100 … 17.000.
 */
object BidRules {

    private const val MIN_PCT = 0.01
    private const val MAX_PCT = 0.20

    fun minPuja(valorBase: Double, ofertaActual: Double): Double =
        ofertaActual + MIN_PCT * valorBase

    fun maxPuja(valorBase: Double, ofertaActual: Double): Double =
        ofertaActual + MAX_PCT * valorBase

    sealed class BidValidation {
        object Ok : BidValidation()
        data class TooLow(val minimo: Double, val mensaje: String) : BidValidation()
        data class TooHigh(val maximo: Double, val mensaje: String) : BidValidation()
        data class Invalid(val mensaje: String) : BidValidation()
    }

    fun validatePuja(
        importe: Double?,
        valorBase: Double,
        ofertaActual: Double,
        categoria: Categoria,
        currency: Currency
    ): BidValidation {
        if (importe == null || importe ≤ 0.0) {
            return BidValidation.Invalid("Ingresá un monto válido.")
        }

        if (categoria.exentaDeLimites) {
            return if (importe > ofertaActual) {
                BidValidation.Ok
            } else {
                BidValidation.TooLow(
                    minimo = ofertaActual,
                    mensaje = "La puja debe ser mayor a ${formatMoney(ofertaActual, currency)}."
                )
```

```
            }
        }

        val min = minPuja(valorBase, ofertaActual)
        val max = maxPuja(valorBase, ofertaActual)

        return when {
            importe < min -> BidValidation.TooLow(
                minimo = min,
                mensaje = "La puja mínima es ${formatMoney(min, currency)}."
            )
            importe > max -> BidValidation.TooHigh(
                maximo = max,
                mensaje = "La puja máxima es ${formatMoney(max, currency)}."
            )
            else -> BidValidation.Ok
        }
    }
}
```

9.3 main/java/com/subastasapp/data/repository/AuctionRepository.kt

```
package com.subastasapp.data.repository

import com.subastasapp.data.model.auction.Auction
import com.subastasapp.data.model.auction.Bid
import com.subastasapp.data.model.auction.BidConfirmation
import com.subastasapp.data.model.auction.Categoria
import com.subastasapp.data.model.auction.CatalogItem

/**
 * Acceso a subastas, catálogo y pujas.
 * Punto de inserción para la futura integración con el backend (Supabase + REST/STOMP).
 */
interface AuctionRepository {
    suspend fun getActiveAuctions(categoria: Categoria? = null): List<Auction>
    suspend fun getAuction(id: Long): Auction?
    suspend fun getCatalogItem(id: Long): CatalogItem?
    suspend fun getBidHistory(auctionId: Long): List<Bid>
    suspend fun placeBid(auctionId: Long, importe: Double, autor: String): Result<BidConfirmation>
}
```

9.4 main/java/com/subastasapp/data/repository/MockAuctionRepository.kt

```
package com.subastasapp.data.repository

import com.subastasapp.data.model.auction.ArtMetadata
import com.subastasapp.data.model.auction.Auction
import com.subastasapp.data.model.auction.AuctionStatus
import com.subastasapp.data.model.auction.Bid
import com.subastasapp.data.model.auction.BidConfirmation
import com.subastasapp.data.model.auction.Categoria
import com.subastasapp.data.model.auction.CatalogItem
import com.subastasapp.data.model.auction.Currency
import com.subastasapp.domain.BidRules
import kotlinx.coroutines.delay
import kotlin.math.ceil

/**
 * Implementación en memoria de [AuctionRepository] para esta entrega (sin backend).
 * Mantiene estado mutable por subasta (oferta actual + historial).
 *
 *  - Subasta #1 (ESPECIAL, ARS): límites de % normales.
 *  - Subasta #2 (ORO, USD): exenta de límites + moneda USD.
 *  - Subasta #3 (PLATINO, ARS): categoría mayor a la del usuario mock → solo lectura.
 *  - Subasta #4 (ESPECIAL, ARS): pieza multi-elemento (juego de té).
 */
object MockAuctionRepository : AuctionRepository {

    private val items = listOf(
        CatalogItem(
            id = 1, numeroPieza = "#88", titulo = "Reloj Rolex GMT-Master II",
            descripcion = "Edición 2023, caja de acero Oystersteel, bisel Cerachrom rojo y azul. " +
                "Una de las piezas más codiciadas de la relojería deportiva contemporánea.",
            valorBase = 15000.0, duenioActual = "Colección privada",
            imagenes = listOf(
                "https://picsum.photos/seed/rolex1/600/400",
                "https://picsum.photos/seed/rolex2/600/400",
                "https://picsum.photos/seed/rolex3/600/400"
            )
        ),
        CatalogItem(
            id = 2, numeroPieza = "#42", titulo = "Reloj Patek Philippe Nautilus",
            descripcion = "Patek Philippe Nautilus en oro rosa. Movimiento automático de manufactura.
            valorBase = 80000.0, duenioActual = "Patrimonio familiar Rossi",
            imagenes = listOf(
                "https://picsum.photos/seed/patek1/600/400",
                "https://picsum.photos/seed/patek2/600/400"
            )
        ),
        CatalogItem(
            id = 3, numeroPieza = "#07", titulo = "Naturaleza",
            descripcion = "Óleo sobre lienzo. Estudio de paisaje de la serie de jardines.",
            valorBase = 15600.0, duenioActual = "Galería Norte",
            imagenes = listOf("https://picsum.photos/seed/monet1/600/400"),
            arte = ArtMetadata(
                artista = "Claude Monet", fecha = "1899",
```

```
                historia = "Pieza atribuida al período de Giverny. Procedencia documentada desde " +
                    "1923, exhibida en dos retrospectivas europeas."
            )
        ),
        CatalogItem(
            id = 4, numeroPieza = "#15", titulo = "Juego de Té de Plata",
            descripcion = "Juego de té victoriano de plata 925, compuesto por 18 piezas.",
            valorBase = 9000.0, duenioActual = "Sucesión Álvarez",
            imagenes = listOf("https://picsum.photos/seed/tea1/600/400"),
            esMultiElemento = true, cantidadElementos = 18
        )
    )

    private val auctions: List<Auction> = listOf(
        Auction(1, "Subasta de Relojes de Lujo", Categoria.ESPECIAL, AuctionStatus.EN_VIVO,
            Currency.ARS, "Mtro. Gómez", "Hoy 20:00", items[0], 15500.0),
        Auction(2, "Alta Relojería Internacional", Categoria.ORO, AuctionStatus.EN_VIVO,
            Currency.USD, "Mtra. Lombardi", "Hoy 21:30", items[1], 85000.0),
        Auction(3, "Arte Moderno y Contemporáneo", Categoria.PLATINO, AuctionStatus.EN_VIVO,
            Currency.ARS, "Mtro. Funes", "Mañana 19:00", items[2], 16000.0),
        Auction(4, "Antigüedades y Platería", Categoria.ESPECIAL, AuctionStatus.EN_VIVO,
            Currency.ARS, "Mtra. Paz", "Hoy 18:00", items[3], 9000.0)
    )

    private class State(var ofertaActual: Double, val history: MutableList<Bid>, var nextBidId: Long)

    private val nombresCompetencia = listOf("Mariana", "Lucas", "Sofía", "Diego", "Valentina", "Martín

    private val state: MutableMap<Long, State> = auctions.associate { a ->
        val seed = mutableListOf(Bid(1, "Subastador", a.itemActual.valorBase, esGanadora = false))
        if (a.ofertaActual > a.itemActual.valorBase) {
            seed.add(Bid(2, nombresCompetencia.random(), a.ofertaActual, esGanadora = true))
        } else {
            seed[0] = seed[0].copy(esGanadora = true)
        }
        a.id to State(a.ofertaActual, seed, nextBidId = seed.size.toLong() + 1)
    }.toMutableMap()

    fun ofertaActual(auctionId: Long): Double = state[auctionId]?.ofertaActual ?: 0.0

    override suspend fun getActiveAuctions(categoria: Categoria?): List<Auction> =
        auctions.filter { it.estado ≠ AuctionStatus.FINALIZADA }
            .filter { categoria == null || it.categoria == categoria }
            .map { it.copy(ofertaActual = ofertaActual(it.id)) }

    override suspend fun getAuction(id: Long): Auction? =
        auctions.find { it.id == id }?.copy(ofertaActual = ofertaActual(id))

    override suspend fun getCatalogItem(id: Long): CatalogItem? = items.find { it.id == id }

    override suspend fun getBidHistory(auctionId: Long): List<Bid> =
        state[auctionId]?.history?.toList() ?: emptyList()

    override suspend fun placeBid(auctionId: Long, importe: Double, autor: String): Result<BidConfirma
        val st = state[auctionId] ?: return Result.failure(IllegalStateException("Subasta inexistente'
        delay(900) // confirmación del servidor: hasta no recibirla, la UI bloquea otra puja
        st.history.replaceAll { it.copy(esGanadora = false) }
```

```
        val bid = Bid(st.nextBidId++, autor, importe, esGanadora = true, esMia = true)
        st.history.add(bid)
        st.ofertaActual = importe
        return Result.success(BidConfirmation(bid, importe))
    }

    /** Puja simulada de la competencia (la usa MockBidStream para mover la subasta en vivo). */
    fun simulateCompetingBid(auctionId: Long): Bid? {
        val auction = auctions.find { it.id == auctionId } ?: return null
        val st = state[auctionId] ?: return null
        val base = auction.itemActual.valorBase
        // La competencia "se rinde" cuando el precio sube demasiado (gana el último postor).
        if (st.ofertaActual ≥ base * 1.35) return null
        val importe = if (auction.categoria.exentaDeLimites) ceil(st.ofertaActual * 1.02)
        else ceil(BidRules.minPuja(base, st.ofertaActual))
        st.history.replaceAll { it.copy(esGanadora = false) }
        val bid = Bid(st.nextBidId++, nombresCompetencia.random(), importe, esGanadora = true, esMia =
        st.history.add(bid)
        st.ofertaActual = importe
        return bid
    }
}
```

9.5 main/java/com/subastasapp/data/repository/CurrentUserProvider.kt

```
package com.subastasapp.data.repository

import com.subastasapp.data.model.auction.Categoria
import com.subastasapp.data.model.auction.CurrentUser

/**
 * Provee el usuario actual para el gating de pujas (mock de esta entrega).
 * Usuario mock: categoría ORO + medio de pago verificado ⇒ puede pujar en ESPECIAL y ORO,
 * solo ver en PLATINO. Con login real, se alimentará del perfil autenticado.
 */
object CurrentUserProvider {

    private val registeredUser = CurrentUser(
        id = 1,
        nombre = "Joaquín",
        categoria = Categoria.ORO,
        tieneMedioPagoVerificado = true,
        registrado = true
    )

    fun current(isGuest: Boolean): CurrentUser =
        if (isGuest) CurrentUser.GUEST else registeredUser
}
```

9.6 main/java/com/subastasapp/data/realtime/BidStream.kt

```kotlin
package com.subastasapp.data.realtime

import com.subastasapp.data.model.auction.Bid
import kotlinx.coroutines.flow.Flow

/**
 * Fuente de pujas en tiempo real para una subasta.
 * Hoy la implementa MockBidStream; con backend la reemplaza un StompBidStream (WebSocket).
 */
interface BidStream {
    fun observeBids(auctionId: Long): Flow<Bid>
}
```

9.7 main/java/com/subastasapp/data/realtime/MockBidStream.kt

```kotlin
package com.subastasapp.data.realtime

import com.subastasapp.data.model.auction.Bid
import com.subastasapp.data.repository.MockAuctionRepository
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flow

/**
 * Implementación mock de [BidStream]: cada cierto tiempo genera una puja de la competencia,
 * simulando el comportamiento en tiempo real que proveerá un WebSocket/STOMP.
 */
object MockBidStream : BidStream {

    private const val INTERVALO_MS = 9000L

    override fun observeBids(auctionId: Long): Flow<Bid> = flow {
        while (true) {
            delay(INTERVALO_MS)
            val bid = MockAuctionRepository.simulateCompetingBid(auctionId)
            if (bid ≠ null) emit(bid)
        }
    }
}
```

9.8 main/java/com/subastasapp/ui/components/CommonUi.kt

```
package com.subastasapp.ui.components

import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.subastasapp.data.model.auction.AuctionStatus
import com.subastasapp.data.model.auction.Categoria
import com.subastasapp.data.model.auction.Currency
import com.subastasapp.data.model.auction.formatMoney
import com.subastasapp.ui.theme.Accent
import com.subastasapp.ui.theme.Primary
import com.subastasapp.ui.theme.TextPrimary

/** Texto monetario formateado según la moneda de la subasta. */
@Composable
fun MoneyText(
    amount: Double,
    currency: Currency,
    fontSize: Int = 16,
    color: Color = TextPrimary,
    weight: FontWeight = FontWeight.Bold
) {
    Text(text = formatMoney(amount, currency), fontSize = fontSize.sp, fontWeight = weight, color = c
}

/** Chip de estado de la subasta (EN VIVO / PROGRAMADA / FINALIZADA). */
@Composable
fun StatusBadge(estado: AuctionStatus, modifier: Modifier = Modifier) {
    val (label, bg, fg) = when (estado) {
        AuctionStatus.EN_VIVO -> Triple("EN VIVO", Color(0xFFFDECEA), Color(0xFFD32F2F))
        AuctionStatus.PROGRAMADA -> Triple("PROGRAMADA", Color(0xFFFFF6E0), Color(0xFF8A6D1B))
        AuctionStatus.FINALIZADA -> Triple("FINALIZADA", Color(0xFFE9E9E9), Color(0xFF555555))
    }
    Surface(color = bg, shape = RoundedCornerShape(4.dp), modifier = modifier) {
        Text(label, color = fg, fontSize = 9.sp, fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(horizontal = 8.dp, vertical = 3.dp))
    }
}

/** Chip de categoría de la subasta. */
@Composable
fun CategoriaBadge(categoria: Categoria, modifier: Modifier = Modifier) {
    Surface(color = Primary, shape = RoundedCornerShape(4.dp), modifier = modifier) {
        Text(categoria.displayName.uppercase(), color = Accent, fontSize = 9.sp, fontWeight = FontWeig
            modifier = Modifier.padding(horizontal = 8.dp, vertical = 3.dp))
```

```
    }
}
```

9.9 main/java/com/subastasapp/ui/auction/AuctionListViewModel.kt

```
package com.subastasapp.ui.auction

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.subastasapp.data.model.auction.Auction
import com.subastasapp.data.model.auction.Categoria
import com.subastasapp.data.repository.AuctionRepository
import com.subastasapp.data.repository.MockAuctionRepository
import com.subastasapp.ui.auth.UiState
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch

class AuctionListViewModel(
    private val repository: AuctionRepository = MockAuctionRepository
) : ViewModel() {

    private val _state = MutableStateFlow<UiState<List<Auction>>>(UiState.Loading)
    val state: StateFlow<UiState<List<Auction>>> = _state.asStateFlow()

    private val _selectedCategoria = MutableStateFlow<Categoria?>(null)
    val selectedCategoria: StateFlow<Categoria?> = _selectedCategoria.asStateFlow()

    init { load() }

    fun selectCategoria(categoria: Categoria?) {
        _selectedCategoria.value = categoria
        load()
    }

    fun load() {
        viewModelScope.launch {
            _state.value = UiState.Loading
            try {
                _state.value = UiState.Success(repository.getActiveAuctions(_selectedCategoria.value))
            } catch (e: Exception) {
                _state.value = UiState.Error("No se pudieron cargar las subastas")
            }
        }
    }
}
```

9.10 main/java/com/subastasapp/ui/auction/AuctionListScreen.kt

```
package com.subastasapp.ui.auction

import androidx.compose.foundation.BorderStroke
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Notifications
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.viewmodel.compose.viewModel
import coil.compose.AsyncImage
import com.subastasapp.GenericViewModelFactory
import com.subastasapp.data.model.auction.Auction
import com.subastasapp.data.model.auction.AuctionStatus
import com.subastasapp.data.model.auction.Categoria
import com.subastasapp.ui.auth.UiState
import com.subastasapp.ui.components.CategoriaBadge
import com.subastasapp.ui.components.MoneyText
import com.subastasapp.ui.components.StatusBadge

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun AuctionListScreen(
    isGuest: Boolean,
    onAuctionClick: (Long) -> Unit,
    viewModel: AuctionListViewModel = viewModel(factory = GenericViewModelFactory { AuctionListViewMod
) {
    val state by viewModel.state.collectAsState()
    val selected by viewModel.selectedCategoria.collectAsState()

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Subasta APP", fontWeight = FontWeight.Bold, color = Color(0xFF1E1E1E))
                actions = {
                    IconButton(onClick = { /* Notificaciones (fuera de alcance) */ }) {
                        Icon(Icons.Default.Notifications, contentDescription = null, tint = Color(0xFF
                    }
                },
```

```
                colors = TopAppBarDefaults.topAppBarColors(containerColor = Color.White)
            )
        },
        containerColor = Color.White
    ) { padding ->
        Column(modifier = Modifier.fillMaxSize().padding(padding)) {
            Text("Subastas activas", fontSize = 22.sp, fontWeight = FontWeight.Bold,
                color = Color(0xFF1E1E1E),
                modifier = Modifier.padding(start = 16.dp, top = 8.dp, bottom = 4.dp))

            CategoryFilterRow(selected = selected, onSelect = viewModel::selectCategoria)

            when (val s = state) {
                is UiState.Loading -> Box(Modifier.fillMaxSize(), Alignment.Center) {
                    CircularProgressIndicator(color = Color(0xFFC9A84C))
                }
                is UiState.Error -> Box(Modifier.fillMaxSize(), Alignment.Center) {
                    Column(horizontalAlignment = Alignment.CenterHorizontally) {
                        Text(s.message, color = Color.Gray)
                        Spacer(Modifier.height(8.dp))
                        TextButton(onClick = viewModel::load) { Text("Reintentar", color = Color(0xFF(
                    }
                }
                is UiState.Success -> {
                    if (s.data.isEmpty()) {
                        Box(Modifier.fillMaxSize(), Alignment.Center) {
                            Text("No hay subastas en esta categoría", color = Color.Gray)
                        }
                    } else {
                        LazyColumn(contentPadding = PaddingValues(16.dp),
                            verticalArrangement = Arrangement.spacedBy(12.dp)) {
                            items(s.data, key = { it.id }) { auction ->
                                AuctionCard(auction = auction, isGuest = isGuest,
                                    onClick = { onAuctionClick(auction.id) })
                            }
                        }
                    }
                }
                else -> Unit
            }
        }
    }
}

@Composable
private fun CategoryFilterRow(selected: Categoria?, onSelect: (Categoria?) -> Unit) {
    Row(modifier = Modifier.fillMaxWidth().horizontalScroll(rememberScrollState())
        .padding(horizontal = 16.dp, vertical = 8.dp),
        horizontalArrangement = Arrangement.spacedBy(8.dp)) {
        FilterPill(label = "Todas", active = selected == null) { onSelect(null) }
        Categoria.values().forEach { cat ->
            FilterPill(label = cat.displayName, active = selected == cat) { onSelect(cat) }
        }
    }
}

@Composable
```

```
private fun FilterPill(label: String, active: Boolean, onClick: () → Unit) {
    Surface(shape = RoundedCornerShape(50),
        color = if (active) Color(0xFF0A1931) else Color.White,
        border = BorderStroke(1.dp, if (active) Color(0xFF0A1931) else Color(0xFFE9E9E9)),
        modifier = Modifier.clickable { onClick() }) {
        Text(label, color = if (active) Color.White else Color(0xFF555555),
            fontSize = 13.sp, fontWeight = FontWeight.Medium,
            modifier = Modifier.padding(horizontal = 16.dp, vertical = 8.dp))
    }
}

@Composable
fun AuctionCard(auction: Auction, isGuest: Boolean, onClick: () → Unit, modifier: Modifier = Modifier
    Card(modifier = modifier.fillMaxWidth().clickable { onClick() },
        colors = CardDefaults.cardColors(containerColor = Color.White),
        border = BorderStroke(1.dp, Color(0xFFE9E9E9)), shape = RoundedCornerShape(12.dp)) {
        Column {
            Box {
                AsyncImage(model = auction.itemActual.imagenes.firstOrNull(),
                    contentDescription = auction.itemActual.titulo,
                    modifier = Modifier.fillMaxWidth().height(160.dp)
                        .clip(RoundedCornerShape(topStart = 12.dp, topEnd = 12.dp))
                        .background(Color(0xFFE9E9E9)),
                    contentScale = ContentScale.Crop)
                Row(modifier = Modifier.padding(8.dp), horizontalArrangement = Arrangement.spacedBy(6
                    StatusBadge(auction.estado)
                    CategoriaBadge(auction.categoria)
                }
            }
            Column(modifier = Modifier.padding(12.dp)) {
                Text(auction.itemActual.titulo, fontSize = 16.sp, fontWeight = FontWeight.Bold,
                    color = Color(0xFF1E1E1E))
                Text("${auction.rematador} · ${auction.fechaHoraInicio}", fontSize = 12.sp,
                    color = Color(0xFF555555))
                Spacer(Modifier.height(8.dp))
                Row(modifier = Modifier.fillMaxWidth(), verticalAlignment = Alignment.CenterVertically
                    horizontalArrangement = Arrangement.SpaceBetween) {
                    Column {
                        Text(if (auction.estado == AuctionStatus.EN_VIVO) "Oferta actual" else "Preci
                            fontSize = 11.sp, color = Color(0xFF555555))
                        // Regla TPO: solo los usuarios registrados ven los precios.
                        if (isGuest) {
                            Text("Iniciá sesión para ver", fontSize = 14.sp, color = Color(0xFF555555
                                fontWeight = FontWeight.Medium)
                        } else {
                            MoneyText(auction.ofertaActual, auction.currency, fontSize = 18, color = (
                        }
                    }
                    Surface(color = Color(0xFFC9A84C), shape = RoundedCornerShape(50)) {
                        Text(if (auction.estado == AuctionStatus.EN_VIVO) "Ver en vivo" else "Ver deta
                            color = Color.White, fontWeight = FontWeight.Bold, fontSize = 12.sp,
                            modifier = Modifier.padding(horizontal = 16.dp, vertical = 8.dp))
                    }
                }
            }
        }
```

```
    }
}
```

9.11 main/java/com/subastasapp/ui/auction/CatalogItemDetailScreen.kt

```
package com.subastasapp.ui.auction

import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.ExpandLess
import androidx.compose.material.icons.filled.ExpandMore
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import coil.compose.AsyncImage
import com.subastasapp.data.model.auction.Auction
import com.subastasapp.data.repository.MockAuctionRepository
import com.subastasapp.ui.components.CategoriaBadge
import com.subastasapp.ui.components.MoneyText
import com.subastasapp.ui.components.StatusBadge

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CatalogItemDetailScreen(
    auctionId: Long,
    isGuest: Boolean,
    onVerEnVivo: (Long) -> Unit,
    onBack: () -> Unit
) {
    val auction by produceState<Auction?>(initialValue = null, auctionId) {
        value = MockAuctionRepository.getAuction(auctionId)
    }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Detalle de la pieza", fontWeight = FontWeight.Bold, color = Color(0xFF
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(Icons.Filled.ArrowBack, contentDescription = "Volver", tint = Color(0xFF1
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(containerColor = Color.White)
```

```
            )
        },
        containerColor = Color.White
    ) { padding ->
        val a = auction
        if (a == null) {
            Box(Modifier.fillMaxSize().padding(padding), Alignment.Center) {
                CircularProgressIndicator(color = Color(0xFFC9A84C))
            }
            return@Scaffold
        }

        val item = a.itemActual
        Column(modifier = Modifier.fillMaxSize().padding(padding)
            .verticalScroll(rememberScrollState()).padding(bottom = 24.dp)) {
            // Carrusel de imágenes
            LazyRow(contentPadding = PaddingValues(horizontal = 16.dp),
                horizontalArrangement = Arrangement.spacedBy(8.dp),
                modifier = Modifier.padding(top = 8.dp)) {
                items(item.imagenes) { url ->
                    AsyncImage(model = url, contentDescription = item.titulo,
                        modifier = Modifier.width(280.dp).height(200.dp).clip(RoundedCornerShape(12.dp
                            .background(Color(0xFFE9E9E9)),
                        contentScale = ContentScale.Crop)
                }
            }

            Column(modifier = Modifier.padding(16.dp)) {
                Row(horizontalArrangement = Arrangement.spacedBy(6.dp)) {
                    StatusBadge(a.estado); CategoriaBadge(a.categoria)
                }
                Spacer(Modifier.height(8.dp))
                Text("Pieza ${item.numeroPieza}", fontSize = 12.sp, color = Color(0xFF555555))
                Text(item.titulo, fontSize = 22.sp, fontWeight = FontWeight.Bold, color = Color(0xFF1E
                Spacer(Modifier.height(4.dp))
                Text("Subasta: ${a.titulo} · ${a.rematador}", fontSize = 12.sp, color = Color(0xFF555
                Text("Inicio: ${a.fechaHoraInicio}", fontSize = 12.sp, color = Color(0xFF555555))

                if (item.esMultiElemento) {
                    Spacer(Modifier.height(8.dp))
                    Surface(color = Color(0xFFF8F9FD), shape = RoundedCornerShape(8.dp)) {
                        Text("Pieza compuesta por ${item.cantidadElementos} elementos",
                            fontSize = 12.sp, color = Color(0xFF0A1931), fontWeight = FontWeight.Mediu
                            modifier = Modifier.padding(horizontal = 12.dp, vertical = 6.dp))
                    }
                }

                Spacer(Modifier.height(16.dp))
                Text("Descripción", fontSize = 14.sp, fontWeight = FontWeight.Bold, color = Color(0xFF
                Spacer(Modifier.height(4.dp))
                Text(item.descripcion, fontSize = 14.sp, color = Color(0xFF555555))

                Spacer(Modifier.height(16.dp))
                Text("Dueño actual", fontSize = 14.sp, fontWeight = FontWeight.Bold, color = Color(0xF
                Text(item.duenioActual, fontSize = 14.sp, color = Color(0xFF555555))

                // Regla TPO: el precio base solo es visible para usuarios registrados.
```

```
                Spacer(Modifier.height(16.dp))
                Text("Precio base", fontSize = 14.sp, fontWeight = FontWeight.Bold, color = Color(0xFF
                if (isGuest) {
                    Text("Iniciá sesión para ver el precio base", fontSize = 14.sp,
                        color = Color(0xFF555555), fontWeight = FontWeight.Medium)
                } else {
                    MoneyText(item.valorBase, a.currency, fontSize = 20, color = Color(0xFFC9A84C))
                }

                item.arte?.let { arte ->
                    Spacer(Modifier.height(16.dp))
                    ArtHistorySection(arte.artista, arte.fecha, arte.historia)
                }

                Spacer(Modifier.height(24.dp))
                Button(onClick = { onVerEnVivo(a.id) },
                    modifier = Modifier.fillMaxWidth().height(52.dp),
                    colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFC9A84C)),
                    shape = RoundedCornerShape(50.dp)) {
                    Text("Ver subasta en vivo", color = Color.White, fontWeight = FontWeight.Bold)
                }
            }
        }
    }
}

@Composable
private fun ArtHistorySection(artista: String, fecha: String, historia: String) {
    var expanded by remember { mutableStateOf(false) }
    Column {
        Row(modifier = Modifier.fillMaxWidth().clickable { expanded = !expanded },
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween) {
            Text("Historial de la pieza", fontSize = 14.sp, fontWeight = FontWeight.Bold, color = Colo
            Icon(if (expanded) Icons.Default.ExpandLess else Icons.Default.ExpandMore,
                contentDescription = null, tint = Color(0xFF555555))
        }
        Text("$artista · $fecha", fontSize = 13.sp, color = Color(0xFFC9A84C), fontWeight = FontWeight
        if (expanded) {
            Spacer(Modifier.height(4.dp))
            Text(historia, fontSize = 14.sp, color = Color(0xFF555555))
        }
    }
}
```

9.12 main/java/com/subastasapp/ui/auction/LiveAuctionViewModel.kt

```
package com.subastasapp.ui.auction

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.subastasapp.data.model.auction.Auction
import com.subastasapp.data.model.auction.Bid
import com.subastasapp.data.model.auction.BiddingStatus
import com.subastasapp.data.model.auction.CurrentUser
import com.subastasapp.data.realtime.BidStream
import com.subastasapp.data.realtime.MockBidStream
import com.subastasapp.data.repository.AuctionRepository
import com.subastasapp.data.repository.CurrentUserProvider
import com.subastasapp.data.repository.MockAuctionRepository
import com.subastasapp.domain.BidRules
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch

/** Estado de la pantalla de subasta en vivo. */
data class LiveAuctionUiState(
    val loading: Boolean = true,
    val auction: Auction? = null,
    val historial: List<Bid> = emptyList(),
    val ofertaActual: Double = 0.0,
    val rangoMin: Double = 0.0,
    val rangoMax: Double? = null,   // null = subasta exenta de límite máximo (ORO/PLATINO)
    val status: BiddingStatus? = null,
    val canBid: Boolean = false,
    val accessReason: String? = null,
    val isBidPending: Boolean = false,
    val inputError: String? = null,
    val finalizada: Boolean = false
)

class LiveAuctionViewModel(
    private val auctionId: Long,
    private val isGuest: Boolean,
    private val repository: AuctionRepository = MockAuctionRepository,
    private val bidStream: BidStream = MockBidStream
) : ViewModel() {

    private val _state = MutableStateFlow(LiveAuctionUiState())
    val state: StateFlow<LiveAuctionUiState> = _state.asStateFlow()

    private val user: CurrentUser = CurrentUserProvider.current(isGuest)
    private var haParticipado = false
    private var winWatch: Job? = null
    private var streamJob: Job? = null

    init { load() }
```

```
    private fun load() {
        viewModelScope.launch {
            val auction = repository.getAuction(auctionId)
            if (auction == null) { _state.value = _state.value.copy(loading = false); return@launch }
            val historial = repository.getBidHistory(auctionId)
            val oferta = auction.ofertaActual
            val (canBid, reason) = evaluarAcceso(auction)

            _state.value = _state.value.copy(
                loading = false, auction = auction, historial = historial, ofertaActual = oferta,
                rangoMin = rangoMinDe(auction, oferta), rangoMax = rangoMaxDe(auction, oferta),
                canBid = canBid, accessReason = reason,
                status = if (!canBid) BiddingStatus.VIEW_ONLY else null
            )
            observarPujas()
        }
    }

    /** Regla TPO: registrado + categoría suficiente + medio de pago verificado. */
    private fun evaluarAcceso(auction: Auction): Pair<Boolean, String?> {
        if (!user.registrado) return false to "Iniciá sesión para poder pujar."
        if (!user.categoria.puedeAcceder(auction.categoria))
            return false to "Tu categoría (${user.categoria.displayName}) no habilita una subasta " +
                "${auction.categoria.displayName}. Podés ver la subasta pero no pujar."
        if (!user.tieneMedioPagoVerificado)
            return false to "Necesitás al menos un medio de pago verificado para pujar."
        return true to null
    }

    // Subastas exentas (ORO/PLATINO): mínimo = superar la oferta actual, sin máximo.
    private fun rangoMinDe(auction: Auction, oferta: Double): Double =
        if (auction.categoria.exentaDeLimites) oferta
        else BidRules.minPuja(auction.itemActual.valorBase, oferta)

    private fun rangoMaxDe(auction: Auction, oferta: Double): Double? =
        if (auction.categoria.exentaDeLimites) null
        else BidRules.maxPuja(auction.itemActual.valorBase, oferta)

    private fun observarPujas() {
        streamJob?.cancel()
        streamJob = viewModelScope.launch {
            bidStream.observeBids(auctionId).collect { bid ->
                if (_state.value.finalizada) { streamJob?.cancel(); return@collect }
                winWatch?.cancel()
                val s = _state.value
                val auction = s.auction ?: return@collect
                val nuevoStatus = when {
                    !s.canBid -> BiddingStatus.VIEW_ONLY
                    haParticipado -> BiddingStatus.SUPERADO
                    else -> null
                }
                _state.value = s.copy(
                    historial = s.historial.map { it.copy(esGanadora = false) } + bid,
                    ofertaActual = bid.importe,
                    rangoMin = rangoMinDe(auction, bid.importe),
                    rangoMax = rangoMaxDe(auction, bid.importe),
```

```
                    status = nuevoStatus
                )
            }
        }
    }

    fun onAmountChange() {
        if (_state.value.inputError ≠ null) _state.value = _state.value.copy(inputError = null)
    }

    fun placeBid(montoTexto: String) {
        val s = _state.value
        val auction = s.auction ?: return
        if (!s.canBid || s.isBidPending || s.finalizada) return

        val monto = montoTexto.replace(".", "").replace(",", ".").toDoubleOrNull()
        val validation = BidRules.validatePuja(monto, auction.itemActual.valorBase,
            s.ofertaActual, auction.categoria, auction.currency)

        val errorMsg = when (validation) {
            is BidRules.BidValidation.Ok → null
            is BidRules.BidValidation.TooLow → validation.mensaje
            is BidRules.BidValidation.TooHigh → validation.mensaje
            is BidRules.BidValidation.Invalid → validation.mensaje
        }
        if (errorMsg ≠ null) { _state.value = s.copy(inputError = errorMsg); return }

        // Regla TPO: una puja a la vez. Bloqueamos hasta recibir la confirmación.
        _state.value = s.copy(isBidPending = true, inputError = null)
        viewModelScope.launch {
            repository.placeBid(auctionId, monto!!, user.nombre).onSuccess { confirmation →
                haParticipado = true
                val cur = _state.value
                val auc = cur.auction!!
                _state.value = cur.copy(
                    historial = cur.historial.map { it.copy(esGanadora = false) } + confirmation.bid,
                    ofertaActual = confirmation.nuevaOfertaActual,
                    rangoMin = rangoMinDe(auc, confirmation.nuevaOfertaActual),
                    rangoMax = rangoMaxDe(auc, confirmation.nuevaOfertaActual),
                    status = BiddingStatus.GANANDO, isBidPending = false
                )
                vigilarCierre()
            }.onFailure {
                _state.value = _state.value.copy(isBidPending = false,
                    inputError = "No se pudo registrar la puja. Reintentá.")
            }
        }
    }

    /** Si el usuario mantiene la oferta más alta y nadie puja por un tiempo, gana la pieza. */
    private fun vigilarCierre() {
        winWatch?.cancel()
        winWatch = viewModelScope.launch {
            delay(7000)
            val s = _state.value
            if (s.status == BiddingStatus.GANANDO && !s.finalizada) {
                streamJob?.cancel()
```

```
                _state.value = s.copy(finalizada = true, status = BiddingStatus.GANADA)
            }
        }
    }
}
```

9.13 main/java/com/subastasapp/ui/auction/LiveAuctionScreen.kt

```
package com.subastasapp.ui.auction

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.viewmodel.compose.viewModel
import coil.compose.AsyncImage
import com.subastasapp.GenericViewModelFactory
import com.subastasapp.data.model.auction.Bid
import com.subastasapp.data.model.auction.BiddingStatus
import com.subastasapp.data.model.auction.Currency
import com.subastasapp.data.model.auction.formatMoney
import com.subastasapp.ui.components.MoneyText

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun LiveAuctionScreen(
    auctionId: Long,
    isGuest: Boolean,
    onBack: () -> Unit,
    onRequireLogin: () -> Unit
) {
    val viewModel: LiveAuctionViewModel = viewModel(
        key = "live_$auctionId",
        factory = GenericViewModelFactory { LiveAuctionViewModel(auctionId, isGuest) }
    )
    val state by viewModel.state.collectAsState()

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Subasta en vivo", fontWeight = FontWeight.Bold, color = Color(0xFF1E1E
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(Icons.Filled.ArrowBack, contentDescription = "Volver", tint = Color(0xFF
                    }
                },
```

```
                colors = TopAppBarDefaults.topAppBarColors(containerColor = Color.White)
            )
        },
        containerColor = Color(0xFFF8F9FD)
    ) { padding ->
        val auction = state.auction
        if (state.loading || auction == null) {
            Box(Modifier.fillMaxSize().padding(padding), Alignment.Center) {
                if (state.loading) CircularProgressIndicator(color = Color(0xFFC9A84C))
                else Text("Subasta no encontrada", color = Color.Gray)
            }
            return@Scaffold
        }

        val currency = auction.currency
        Column(modifier = Modifier.fillMaxSize().padding(padding)) {
            StatusBanner(state.status, state.accessReason)

            Column(modifier = Modifier.weight(1f).padding(horizontal = 16.dp)) {
                // Artículo actual
                Card(modifier = Modifier.fillMaxWidth().padding(top = 12.dp),
                    colors = CardDefaults.cardColors(containerColor = Color.White),
                    shape = RoundedCornerShape(12.dp)) {
                    Row(modifier = Modifier.padding(12.dp), verticalAlignment = Alignment.CenterVertic
                        AsyncImage(model = auction.itemActual.imagenes.firstOrNull(),
                            contentDescription = auction.itemActual.titulo,
                            modifier = Modifier.size(72.dp).clip(RoundedCornerShape(8.dp)).background(
                            contentScale = ContentScale.Crop)
                        Spacer(Modifier.width(12.dp))
                        Column(modifier = Modifier.weight(1f)) {
                            Text("Artículo actual", fontSize = 11.sp, color = Color(0xFF555555))
                            Text(auction.itemActual.titulo, fontSize = 16.sp, fontWeight = FontWeight
                                color = Color(0xFF1E1E1E))
                            Text("Pieza ${auction.itemActual.numeroPieza}", fontSize = 12.sp, color =
                        }
                    }
                }

                Spacer(Modifier.height(12.dp))
                Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(1
                    InfoBox(Modifier.weight(1f), "Oferta actual", formatMoney(state.ofertaActual, curr
                    InfoBox(Modifier.weight(1f), "Rango de puja válido",
                        rangoText(state.rangoMin, state.rangoMax, currency), Color(0xFFC9A84C), small
                }

                Spacer(Modifier.height(16.dp))
                Text("Historial de pujas", fontWeight = FontWeight.Bold, fontSize = 14.sp, color = Col
                Spacer(Modifier.height(4.dp))
                LazyColumn(modifier = Modifier.weight(1f).fillMaxWidth()) {
                    items(state.historial.reversed()) { bid ->
                        BidRow(bid, currency)
                        Divider(color = Color(0xFFE9E9E9))
                    }
                }
            }

            BidControl(
```

```
                canBid = state.canBid, isGuest = isGuest, isBidPending = state.isBidPending,
                finalizada = state.finalizada, inputError = state.inputError,
                accessReason = state.accessReason, currency = currency,
                onAmountChange = viewModel::onAmountChange, onPujar = viewModel::placeBid,
                onRequireLogin = onRequireLogin
            )
        }
    }
}

@Composable
private fun StatusBanner(status: BiddingStatus?, reason: String?) {
    if (status == null) return
    val (text, bg, fg) = when (status) {
        BiddingStatus.GANANDO → Triple("Vas ganando la subasta", Color(0xFFE8F5E9), Color(0xFF2E7D32)
        BiddingStatus.SUPERADO → Triple("Te superaron la oferta", Color(0xFFFDECEA), Color(0xFFD32F2F
        BiddingStatus.GANADA → Triple("¡OFERTA GANADORA! Ganaste la pieza", Color(0xFFC9A84C), Color
        BiddingStatus.VIEW_ONLY → Triple(reason ?: "Modo solo lectura", Color(0xFFEDEDED), Color(0xFF
    }
    Surface(color = bg, modifier = Modifier.fillMaxWidth()) {
        Text(text, color = fg, fontWeight = FontWeight.Bold, fontSize = 13.sp,
            modifier = Modifier.padding(horizontal = 16.dp, vertical = 10.dp))
    }
}

@Composable
private fun InfoBox(modifier: Modifier, label: String, value: String, valueColor: Color, small: Boolea
    Card(modifier = modifier, colors = CardDefaults.cardColors(containerColor = Color.White),
        shape = RoundedCornerShape(8.dp)) {
        Column(modifier = Modifier.padding(12.dp)) {
            Text(label, fontSize = 11.sp, color = Color(0xFF555555))
            Spacer(Modifier.height(2.dp))
            Text(value, fontSize = if (small) 14.sp else 22.sp, fontWeight = FontWeight.Bold, color =
        }
    }
}

@Composable
private fun BidRow(bid: Bid, currency: Currency) {
    Row(modifier = Modifier.fillMaxWidth().padding(vertical = 8.dp),
        horizontalArrangement = Arrangement.SpaceBetween, verticalAlignment = Alignment.CenterVertical
        Column {
            Text(if (bid.esMia) "${bid.autor} (vos)" else bid.autor, fontSize = 14.sp,
                fontWeight = if (bid.esMia) FontWeight.Bold else FontWeight.Normal, color = Color(0xFF
            if (bid.esGanadora) Text("Oferta más alta", fontSize = 11.sp, color = Color(0xFF2E7D32))
        }
        MoneyText(bid.importe, currency, fontSize = 15, color = Color(0xFF0A1931))
    }
}

@Composable
private fun BidControl(
    canBid: Boolean, isGuest: Boolean, isBidPending: Boolean, finalizada: Boolean,
    inputError: String?, accessReason: String?, currency: Currency,
    onAmountChange: () → Unit, onPujar: (String) → Unit, onRequireLogin: () → Unit
) {
    Surface(color = Color.White, shadowElevation = 8.dp) {
```

```
        Column(modifier = Modifier.fillMaxWidth().padding(16.dp)) {
            when {
                finalizada -> Text("La subasta finalizó.", color = Color(0xFF555555),
                    fontWeight = FontWeight.Medium, modifier = Modifier.padding(vertical = 8.dp))
                !canBid -> {
                    Text(accessReason ?: "No podés pujar en esta subasta.", color = Color(0xFF555555)
                    if (isGuest) {
                        Spacer(Modifier.height(8.dp))
                        Button(onClick = onRequireLogin, modifier = Modifier.fillMaxWidth().height(50
                            colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFC9A84C)),
                            shape = RoundedCornerShape(50.dp)) {
                            Text("Iniciá sesión para pujar", color = Color.White, fontWeight = FontWe
                        }
                    }
                }
                else -> {
                    var amount by remember { mutableStateOf("") }
                    if (inputError ≠ null) {
                        Text(inputError, color = Color(0xFFD32F2F), fontSize = 12.sp)
                        Spacer(Modifier.height(4.dp))
                    }
                    Row(verticalAlignment = Alignment.CenterVertically) {
                        OutlinedTextField(value = amount, onValueChange = { amount = it; onAmountChan
                            label = { Text("Monto en ${currency.name}") }, singleLine = true, enabled
                            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
                            isError = inputError ≠ null, modifier = Modifier.weight(1f))
                        Spacer(Modifier.width(8.dp))
                        Button(onClick = { onPujar(amount) }, enabled = !isBidPending,
                            modifier = Modifier.height(56.dp),
                            colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF0A1931)),
                            shape = RoundedCornerShape(12.dp)) {
                            if (isBidPending) CircularProgressIndicator(color = Color.White, strokeWi
                                modifier = Modifier.size(20.dp))
                            else Text("PUJAR", color = Color.White, fontWeight = FontWeight.Bold)
                        }
                    }
                    if (isBidPending) {
                        Spacer(Modifier.height(6.dp))
                        Text("Esperando confirmación del sistema…", fontSize = 11.sp, color = Color(0
                    }
                }
            }
        }
    }
}

private fun rangoText(min: Double, max: Double?, currency: Currency): String =
    if (max == null) "Mayor a ${formatMoney(min, currency)} (sin máx.)"
    else "${formatMoney(min, currency)} – ${formatMoney(max, currency)}"
```

9.14 main/java/com/subastasapp/ui/auction/AuctionPreviews.kt

```
package com.subastasapp.ui.auction

import androidx.compose.foundation.layout.padding
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.subastasapp.data.model.auction.ArtMetadata
import com.subastasapp.data.model.auction.Auction
import com.subastasapp.data.model.auction.AuctionStatus
import com.subastasapp.data.model.auction.Categoria
import com.subastasapp.data.model.auction.CatalogItem
import com.subastasapp.data.model.auction.Currency
import com.subastasapp.ui.theme.SubastasAppTheme

private val sampleItem = CatalogItem(
    id = 1, numeroPieza = "#88", titulo = "Reloj Rolex GMT-Master II",
    descripcion = "Edición 2023, caja de acero Oystersteel.", valorBase = 15000.0,
    duenioActual = "Colección privada",
    arte = ArtMetadata("Rolex", "2023", "Pieza deportiva contemporánea.")
)

private val sampleAuction = Auction(
    id = 1, titulo = "Subasta de Relojes de Lujo", categoria = Categoria.ESPECIAL,
    estado = AuctionStatus.EN_VIVO, currency = Currency.ARS, rematador = "Mtro. Gómez",
    fechaHoraInicio = "Hoy 20:00", itemActual = sampleItem, ofertaActual = 15500.0
)

@Preview(name = "AuctionCard - registrado", showBackground = true)
@Composable
private fun AuctionCardRegisteredPreview() {
    SubastasAppTheme {
        AuctionCard(sampleAuction, isGuest = false, onClick = {}, modifier = Modifier.padding(16.dp))
    }
}

@Preview(name = "AuctionCard - invitado", showBackground = true)
@Composable
private fun AuctionCardGuestPreview() {
    SubastasAppTheme {
        AuctionCard(sampleAuction, isGuest = true, onClick = {}, modifier = Modifier.padding(16.dp))
    }
}
```

9.15 test/java/com/subastasapp/domain/BidRulesTest.kt

```
package com.subastasapp.domain

import com.subastasapp.data.model.auction.Categoria
import com.subastasapp.data.model.auction.Currency
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

class BidRulesTest {

    private val ars = Currency.ARS

    @Test fun `minPuja aplica 1 por ciento del valor base`() {
        assertEquals(15100.0, BidRules.minPuja(10000.0, 15000.0), 0.001)
    }

    @Test fun `maxPuja aplica 20 por ciento del valor base`() {
        assertEquals(17000.0, BidRules.maxPuja(10000.0, 15000.0), 0.001)
    }

    @Test fun `puja por debajo del minimo es rechazada en categoria comun`() {
        assertTrue(BidRules.validatePuja(15050.0, 10000.0, 15000.0, Categoria.COMUN, ars)
                is BidRules.BidValidation.TooLow)
    }

    @Test fun `puja en el limite inferior es valida`() {
        assertTrue(BidRules.validatePuja(15100.0, 10000.0, 15000.0, Categoria.COMUN, ars)
                is BidRules.BidValidation.Ok)
    }

    @Test fun `puja en el limite superior es valida`() {
        assertTrue(BidRules.validatePuja(17000.0, 10000.0, 15000.0, Categoria.COMUN, ars)
                is BidRules.BidValidation.Ok)
    }

    @Test fun `puja por encima del maximo es rechazada en categoria comun`() {
        assertTrue(BidRules.validatePuja(17001.0, 10000.0, 15000.0, Categoria.COMUN, ars)
                is BidRules.BidValidation.TooHigh)
    }

    @Test fun `monto nulo o no positivo es invalido`() {
        assertTrue(BidRules.validatePuja(null, 10000.0, 15000.0, Categoria.COMUN, ars)
                is BidRules.BidValidation.Invalid)
        assertTrue(BidRules.validatePuja(0.0, 10000.0, 15000.0, Categoria.COMUN, ars)
                is BidRules.BidValidation.Invalid)
    }

    @Test fun `categoria oro permite superar el maximo del 20 por ciento`() {
        assertTrue(BidRules.validatePuja(50000.0, 10000.0, 15000.0, Categoria.ORO, ars)
                is BidRules.BidValidation.Ok)
    }

    @Test fun `categoria platino exige solo superar la oferta actual`() {
```

```
        assertTrue(BidRules.validatePuja(15000.0, 10000.0, 15000.0, Categoria.PLATINO, ars)
                is BidRules.BidValidation.TooLow)
        assertTrue(BidRules.validatePuja(15001.0, 10000.0, 15000.0, Categoria.PLATINO, ars)
                is BidRules.BidValidation.Ok)
    }

    @Test fun `puedeAcceder respeta la jerarquia de categorias`() {
        assertTrue(Categoria.ORO.puedeAcceder(Categoria.ESPECIAL))
        assertTrue(Categoria.ORO.puedeAcceder(Categoria.ORO))
        assertTrue(!Categoria.ORO.puedeAcceder(Categoria.PLATINO))
    }
}
```

9.16 Fragmentos de archivos modificados

`main/java/com/subastasapp/ui/main/MainScreen.kt` — tab Inicio con navegación anidada:

```
composable(BottomNavItem.Home.route) {
    // Inicio: navegación anidada Subastas activas → Detalle → Subasta en vivo
    val homeNav = rememberNavController()
    NavHost(navController = homeNav, startDestination = "auctions_list") {
        composable("auctions_list") {
            AuctionListScreen(
                isGuest = isGuest,
                onAuctionClick = { id → homeNav.navigate("auction_detail/$id") }
            )
        }
        composable("auction_detail/{id}") { entry →
            val id = entry.arguments?.getString("id")?.toLongOrNull()
            if (id ≠ null) {
                CatalogItemDetailScreen(
                    auctionId = id, isGuest = isGuest,
                    onVerEnVivo = { aId → homeNav.navigate("auction_live/$aId") },
                    onBack = { homeNav.popBackStack() }
                )
            }
        }
        composable("auction_live/{id}") { entry →
            val id = entry.arguments?.getString("id")?.toLongOrNull()
            if (id ≠ null) {
                LiveAuctionScreen(
                    auctionId = id, isGuest = isGuest,
                    onBack = { homeNav.popBackStack() },
                    onRequireLogin = {
                        rootNavController.navigate("login") { popUpTo(0) { inclusive = true } }
                    }
                )
            }
        }
    }
}
```

`main/java/com/subastasapp/data/api/RetrofitClient.kt` — base URL para emulador:

```
// 10.0.2.2 es el alias del emulador de Android para el localhost de la PC anfitriona.
// (Para un dispositivo físico, reemplazar por la IP LAN de la PC, ej. http://192.168.0.13:8080/)
private const val BASE_URL = "http://10.0.2.2:8080/"
```

`main/java/com/subastasapp/ui/auth/LoginScreen.kt` — botón debug (temporal):

```
// DEBUG: acceso temporal como usuario registrado (categoría ORO, medio de pago
// verificado) sin backend. Quitar cuando exista el login real contra Supabase.
TextButton(onClick = { onEnterAsDemo() }, modifier = Modifier.fillMaxWidth()) {
    Text("🔧 Entrar como usuario demo (debug)", color = Color.Gray, fontSize = 13.sp)
}
```

`main/java/com/subastasapp/navigation/NavGraph.kt` — ruta de la entrada demo (temporal):

```
// DEBUG: entra al flujo de usuario registrado sin backend.
onEnterAsDemo = {
    navController.navigate(Screen.Main.route) {
        popUpTo(Screen.Login.route) { inclusive = false }
    }
}
```